

SDK / стек

Параметр	Значение
Название приложения	Ice Fishing
Android Gradle Plugin	8.11.2
minSdk	24
targetSdk	36
Kotlin	да, 2.0.21
Web View	да
Custom Tabs	да
Рекламные сети	нет
Аналитика	Adjust, Firebase Messaging, Firebase Installations
Permissions	android.permission.INTERNET, android.permission.ACCESS_NETWORK_STATE, com.google.android.gms.permission.AD_ID, android.permission.CAMERA, android.permission.POST_NOTIFICATIONS, android.permission.WAKE_LOCK, com.google.android.c2dm.permission.RECEIVE, com.catchingfish.fishcatcherpro.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE, com.android.vending.CHECK_LICENSE
Libraries	AGP 8.11.2 / Gradle 8.13; Kotlin 2.0.21; kotlinx-coroutines 1.9.0; Jetpack Compose UI 1.9.2 / Animation 1.7.0 / Material3 1.3.0; AndroidX Activity 1.13.0, AppCompat 1.7.1, Core 1.18.0, Lifecycle 2.10.0, Fragment 1.5.4, Browser 1.10.0 (Custom Tabs), DataStore Preferences 1.1.7, Room 2.8.4, Material 1.13.0; OkHttp 3.14.9 (WebSocket/HTTP); Adjust SDK android5.5.0 (com.adjust.sdk, libsigner.so); Firebase Messaging 25.0.1, Firebase Installations, Firebase Firestore, Firebase Common/KTX, Datatransport/CCT; Google Play Services (base/basement, cloud-messaging, ads-identifier, tasks); Play Install Referrer API; Pairip licensecheck / Application wrapper; libnative_params.so (XorDecoder, CFH!)
Подозрительные домены	нет
SharedPreferences	com.catchingfish.fishcatcherpro: ключ url — кэш URL оффера после ответа cloaking-сервера; com.catchingfish.fishcatcherpro_preferences — prefs Application; AdjustIoActivityState / attribution; com.google.android.gms.appid (FCM); FirebaseHeartBeat* / firebase.messaging; DataStore Preferences; WebView CookieManager.flush()
Есть ли клоака	да
Подозрительные слова	postback, postbackAddress, acceptDestination, bridge, WebView, user-agent, encryptionPassword, zeroAdvertisingId, licensecheck, catchFishVector, riverHookPath, waterReelCipher, fishSyncCatch, catchingFishGate

Собираемые данные

- advertisingId / catchFishVector (GAID)
- installReferrer / riverHookPath
- messagingToken / waterReelCipher (FCM)
- adjustId / fishSyncCatch (adid)
- Adjust campaign attribution JSON
- zeroAdvertisingId (00000000-0000-0000-0000-000000000000)
- packageName
- encryptionPassword (catchingFishGate)
- adjustApplicationToken (qz1vn5lokngw)

Как работает клоака

В начале

Снаружи это обычная игра про подлёдную рыбалку (меню, партия, музыка). Обман в том, что при запуске MainActivity сначала спрашивает скрытый сервер: оставить человека в «белой» игре или отправить на веб-оффер. «Нужный» пользователь в итоге попадает на URL оффера (лендинг/кабинет) в WebView или Chrome Custom Tabs (встроенный браузерный экран Chrome). Судьбу решает ответ OkHttp WebSocket (постоянное соединение) на <https://ultraweavekernel.digital/>.

Зачем эти данные

advertisingId → отпечаток устройства для фильтрации аудитории. installReferrer → источник установки из Play (органика vs кампания). messagingToken → FCM-токен как дополнительный идентификатор. adjustId + attribution JSON → атрибуция Adjust: кампания/сеть. См. полный список выше.

Куда отправляется запрос

Собственный шлюз: <https://ultraweavekernel.digital/> через OkHttp WebSocket (клиент также понимает схемы ws/wss и приводит их к http/https). Перед отправкой JSON-пакет шифруется: PBKDF2-HMAC-SHA256 (10000 итераций) + AES-GCM, пароль catchingFishGate из libnative_params.so (XOR-ключ CFH!). На проводе уходит Base64(salt||iv||ciphertext). Если ответ пустой/невалидный — белый путь: MenuGameActivity (локальная игра), без веб-оффера.

Как приходит ответ

Сервер отвечает текстом-адресом (строка URL). Пустая/невалидная строка → белый режим. Непустая http(s)-ссылка → acceptDestination: URL пишется в SharedPreferences (имя файла = package, ключ url) и дальше открывается как оффер.

Как показывается целевая страница

Боевой URL открывают в полноэкранном WebView внутри MainActivity (JS, cookies, file chooser, CAMERA через WebChromeClient) и/или через Chrome Custom Tabs (customTabsCoordinator). Финальный URL берётся из ответа WebSocket-шлюза (или из кэша prefs/url при повторном запуске). Пользователь видит веб-лендинг/оффер, а не рыбалку.

Цепочка по шагам

- Шаг 1. Старт MainActivity → Adjust.initSdk(token qz1vn5lokngw, production).
- Шаг 2. Чтение SharedPreferences/url → если уже есть URL → сразу WebView (без нового probe).
- Шаг 3. Иначе сбор signals: GAID, install referrer, FCM token, Adjust ID + attribution JSON.
- Шаг 4. JSON с именами catchFishVector / riverHookPath / waterReelCipher / fishSyncCatch → шифрование PBKDF2+AES-GCM (catchingFishGate).
- Шаг 5. Отправка Base64-пакета по WebSocket на <https://ultraweavekernel.digital/>.
- Шаг 6. Ответ: пусто → MenuGameActivity (белая игра); http(s) URL → сохранить в prefs/url.
- Шаг 7. acceptDestination → показать URL в WebView и/или Custom Tabs.
- Шаг 8. Повторный вход берёт кэш url и снова открывает оффер без нового запроса.

Роль подозрительных доменов

В основной таблице кастомные хосты не вынесены отдельной VT-проверкой. Рабочий шлюз решения из native-кода — <https://ultraweavekernel.digital/>: командный WebSocket postback (принимает зашифрованный payload и возвращает пусто или URL оффера), а не CDN самого лендинга.

Финал

«Боевой» пользователь оказывается на URL, который вернул ultraweavekernel.digital, внутри WebView/Custom Tabs. Финальный адрес приходит с шлюза и кэшируется в SharedPreferences под ключом url.